

# KCMS Proposal

## KCMS

### Khmer Comment Moderation System

---

**A moderation platform that catches abuse and scams in Khmer Facebook comments while keeping legitimate complaints visible.**

Prepared August 2026 · Prototype running at `kcms-frontend.vercel.app`

---

### Table of contents

---

1. Executive summary
  2. The problem
  3. What KCMS does
  4. System architecture
    - 4.1 The system at a glance
    - 4.2 The runtime path
    - 4.3 Two independent axes
    - 4.4 Routing rules
    - 4.5 The record model
    - 4.6 Storage and the erasure boundary
    - 4.7 The classifier seam
    - 4.8 Two loops, different speeds
  5. Identity, workspaces and authorization
  6. What is built today
  7. The Khmer model path
  8. Measuring honestly
  9. What we deliberately cannot do
  10. Roadmap
  11. Appendix A — Technology
  12. Appendix B — API surface
  13. Appendix C — Accepted decisions
-

## 1. Executive summary

---

Cambodian news outlets, institutions and business pages are flooded with toxic comments, scams and harassment written in Khmer and Khmerlish. Global moderation tools barely read the language, so page admins fall back on keyword blocklists that one changed character defeats, or on manual review that stops scaling at a few hundred comments a day.

Two failures happen at once. **Real threats stay up, and ordinary complaints about public services get deleted alongside them.**

KCMS classifies every comment on two independent axes — how severe it is, and who it is aimed at — then routes what needs attention to a human. Criticism directed at an organisation is never queued for removal, however hostile. A person decides every moderation action.

A working full-stack prototype is deployed today: public site, authentication, isolated client workspaces, a moderation queue against real PostgreSQL, and a platform administration surface. The classifier behind it is deliberately a disclosed, versioned pattern matcher; a trained Khmer model replaces it later through an interface that already exists.

---

## 2. The problem

---

### Who has it

Social media managers and page admins moderating Khmer comment sections every day: news outlets, ministries and public institutions, online sellers running their business through a Facebook Page, and agencies managing client pages.

### What they do today

| Approach                     | Why it fails                                                            |
|------------------------------|-------------------------------------------------------------------------|
| Facebook's keyword blocklist | Matches exact words. One changed character passes. No sense of context. |
| Manual review                | Works, then stops scaling past a few hundred comments a day.            |
| General moderation APIs      | None meaningfully support Khmer.                                        |

### The distinction nobody handles

Spotting profanity is not the hard problem. The hard problem is telling **targeted abuse** apart from **someone legitimately angry about a public service**. Tools built only to remove bad content delete both.

Consider two comments with identical severity:

| Comment                                  | Severity  | Target      | Correct action |
|------------------------------------------|-----------|-------------|----------------|
| "Sopheha is a thief, she stole my money" | Offensive | Person      | Review         |
| "ABA are thieves, they stole my money"   | Offensive | Institution | Leave up       |

A single-axis system routes both toward suppression. That is precisely the error this product exists to avoid, and KCMS treats it as an error to be **measured**, not an acceptable cost.

### 3. What KCMS does



Four things distinguish it:

**Khmer first.** Khmer script, romanised Khmerlish, everyday slang, misspellings and deliberately obfuscated words — the writing people actually use.

**Two axes, not one score.** Automatic hiding, when it eventually exists, is gated on a conjunction across both. A single number cannot express *“certain this is harmful, unsure who it is aimed at.”*

**Explainability.** Every surfaced comment carries the reason it reached a human: possible harm, aimed at an organisation, unfamiliar wording, low confidence. Not a mysterious toxicity score.

**Humans decide.** Detection is automatic. What happens to a comment is not.

## 4. System architecture

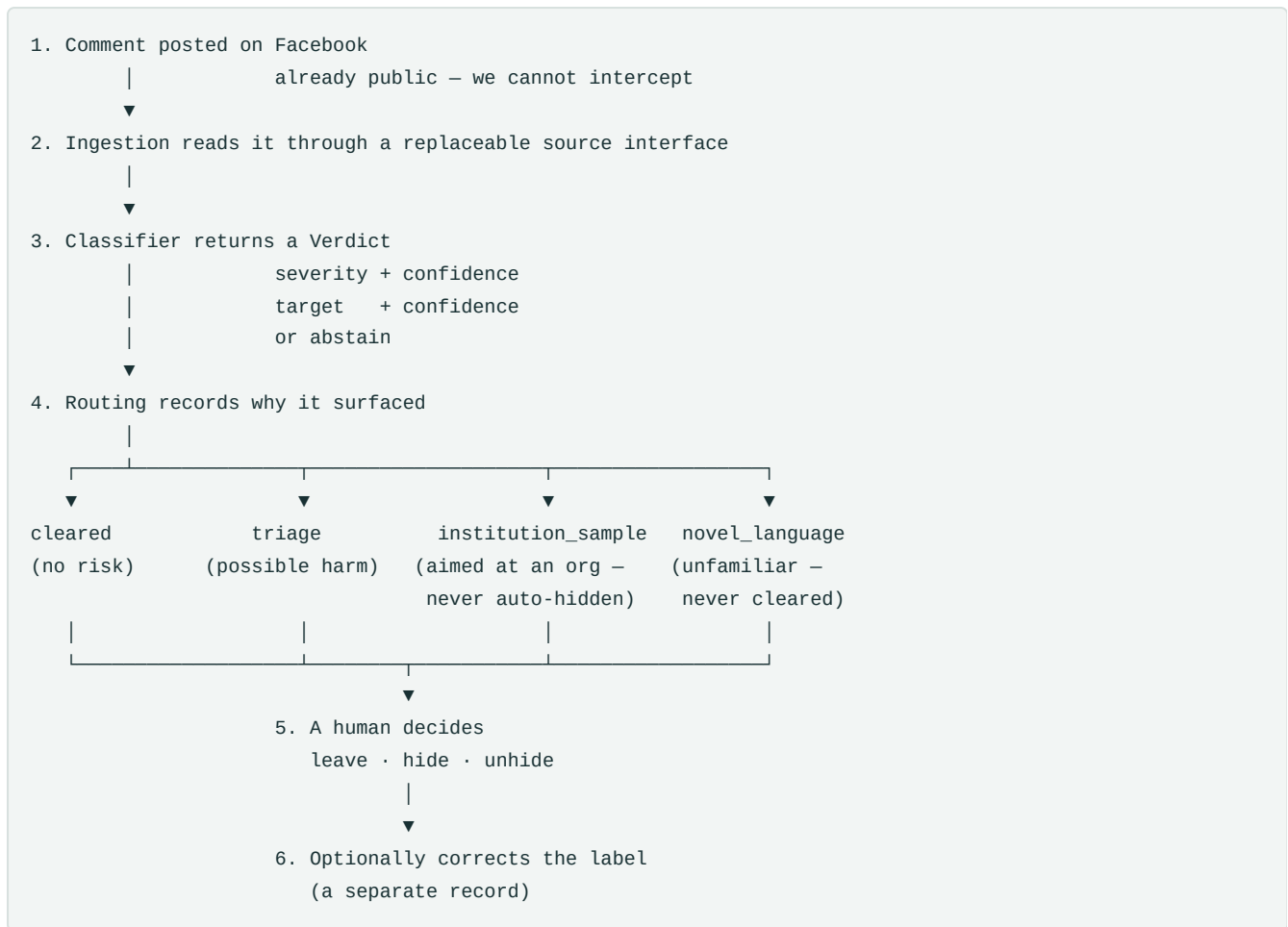
### 4.1 The system at a glance



Frontend and backend are **independent repositories**. The backend owns an OpenAPI 3.1 artifact; the frontend generates its client from it. A test asserts the committed artifact matches the application byte for byte, so contract drift fails the build rather than surfacing as a runtime error.

### 4.2 The runtime path

What happens to a single comment, in order:



**We react, we do not gate.** Facebook publishes a comment before we can see it. There is no pre-publication hook, so latency between publication and action is seconds to minutes and cannot be zero. This is a moderation copilot, not a gateway.

### 4.3 Two independent axes

| Axis     | Values                         |
|----------|--------------------------------|
| Severity | SAFE · OFFENSIVE · HARMFUL     |
| Target   | PERSON · INSTITUTION · NEITHER |

Each carries its own confidence. A Verdict also records an abstention state, a rationale where available, and the version of the model that produced it.

## 4.4 Routing rules

abstain → always a human. No policy overrides this.

target = INSTITUTION → never routed for removal, at any confidence, however hostile.

unrecognised language → abstain and surface. A no-pattern-hit is NOT evidence of safety.

automatic hiding → off. Every action is taken by a person.

The third rule matters more than it looks. A classifier that finds no match cannot distinguish *safe* from *unfamiliar*. Treating both as cleared silently clears exactly the evolving Khmer slang the product exists to catch. Unfamiliar wording therefore surfaces as novel\_language, and becomes a labelled training candidate rather than a silent miss.

## 4.5 The record model

Three record kinds. **Never derived from one another.**

| Record     | Is                                     | Carries                                                          |
|------------|----------------------------------------|------------------------------------------------------------------|
| Verdict    | What the model asserted                | Both labels, both confidences, abstain, rationale, model version |
| Action     | What happened to the comment           | hide / unhide / leave, actor, time                               |
| Correction | What a human says the labels should be | Per-axis, actor, the model version it disagrees with             |

**Hiding a comment writes no Correction.** This is the single most important invariant in the system.

If moderator actions became training labels, the model would drift toward suppression every week — because moderators under volume pressure over-remove — while the dashboard showed *improving* agreement with humans. The product would converge on exactly the failure it was built to avoid, and the metrics would look healthier the whole way down.

Silence is not agreement. Correction volume is low by design; that is the cost of not manufacturing labels nobody gave.

## 4.6 Storage and the erasure boundary

|                            |                                      |
|----------------------------|--------------------------------------|
| <u>comment_content</u>     | <u>verdict · action · correction</u> |
| comment_id                 | comment_id                           |
| workspace_id               | labels, confidences, versions        |
| text                       | actor                                |
| post_text                  | occurred_at                          |
| parent_text                |                                      |
| author_ref                 |                                      |
| purged on delete at source | append-only, never deleted           |

When a commenter deletes their comment on Facebook, the content is purged. The record that a decision was made survives.

**Deleting speech removes the speech; it does not falsify the audit trail.**

## 4.7 The classifier seam

One interface. Two implementations. Nothing downstream knows which is running.

```
@dataclass(frozen=True)
class Verdict:
    severity: Severity          # SAFE | OFFENSIVE | HARMFUL
    severity_confidence: float
    target: Target              # PERSON | INSTITUTION | NEITHER
    target_confidence: float
    abstain: bool
    surfaced_reason: SurfacedReason
    rationale: str | None
    model_version: str

class Classifier(Protocol):
    async def classify(
        self, items: Sequence[CommentContext]
    ) -> Sequence[Verdict]: ...
```

Swapping the trained model in is one line:

```
classifier = PatternMatcher()          # now - disclosed rules, v0.1
classifier = KhmerModel("./model_v1")  # later - fine-tuned transformer
```

Routing, queue, thresholds, metrics and storage all talk to this interface, never to a model.

The classifier running today is **not** Khmer NLP. Every accuracy claim about it is a claim about routing, not about language understanding. It exists so the whole system can be built and proven before the model is ready.

## 4.8 Two loops, different speeds

FAST · continuous

read → classify → route → act → collect

SLOW · every few weeks

export corrections → retrain offline → model file → deploy

They meet at exactly one place: **the model file**.

**Nothing learns while the system is running.** The model is frozen between deployments — same weights, same behaviour, until a new file is deployed. Training happens outside the platform. What returns is a model, not data.

## 5. Identity, workspaces and authorization

### Two visible roles

| Role                   | Manages                                                                  |
|------------------------|--------------------------------------------------------------------------|
| Client                 | Their workspace, Page connection, team, settings, and comment moderation |
| Platform Administrator | Access requests, client workspaces, service health, audited support      |

### Workspace isolation

Every account owns a workspace. Comments, verdicts, actions and corrections are scoped to it. Two accounts never see each other's data or decisions.

A request for a resource in another workspace returns **404, not 403** — a 403 would confirm the resource exists somewhere else.

### Sign-in

Email with scrypt hashing, and Telegram Login Widget with HMAC payload verification. Identity is modelled per provider, so one account can hold both.

Sessions are **bearer tokens**, not cookies: the frontend and API are served from different sites, where SameSite=None cookies are blocked by default in several browsers. Only the SHA-256 of a token is stored, so a database leak does not hand over usable sessions.

## Platform Administration cannot read customer comments

The specification has always stated that Platform Administrators cannot browse customer comment content through ordinary administration views. That rule is now **enforced by test**: the administration response is walked for forbidden keys at every nesting level, so a future “just a small preview” field fails the build rather than quietly handing customer comments to platform staff.

The role itself comes from an environment allowlist reconciled at sign-in. It is never settable through the API, and removing an address actually revokes it.

---

## 6. What is built today

---

Deployed and running against real PostgreSQL.

| Area                                                             | Status    |
|------------------------------------------------------------------|-----------|
| Public site, bilingual Khmer/English                             | Live      |
| Reviewed pilot access, invitation setup, and sign-in             | Live      |
| Isolated client workspaces                                       | Live      |
| Moderation work list, paginated                                  | Live      |
| Actions — leave / hide / unhide, reversible                      | Live      |
| Corrections, separate from actions                               | Live      |
| Direct Client Page connection (Facebook Login or advanced token) | Live      |
| Team membership and invitation links                             | Live      |
| Workspace and account settings                                   | Live      |
| Platform administration — pilot-access review                    | Live      |
| Comment review detail with post and parent context               | Live      |
| Real Facebook ingestion                                          | Not built |
| Trained Khmer model                                              | Not built |

**Verification:** 77 backend tests including integration tests against real PostgreSQL and 34 frontend unit tests pass. Browser verification is repeated for each deployable slice.

---

## 7. The Khmer model path

The prototype exists so the model can be built against a working system rather than in isolation.

|         |                                                                                                                                                                            |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1 | Authorised seed dataset<br>Khmer, Khmerlish, slang, misspellings, obfuscation,<br>person-directed abuse, institutional criticism, safe traffic                             |
| Phase 2 | Manual annotation<br>Written labelling guideline · blind annotators ·<br>two independent labels on evaluation samples ·<br>adjudicated disagreements                       |
| Phase 3 | Dataset quality<br>Deduplicate · remove identifiers · balance categories ·<br>keep one conversation in one split · version every release                                   |
| Phase 4 | Baseline model<br>Severity + target + independent confidences + abstention.<br>It does not hide anything.                                                                  |
| Phase 5 | Shadow mode<br>Classifies live traffic. Clients still decide everything.<br>Corrections become curated training candidates.                                                |
| Phase 6 | Controlled improvement<br>Retrain offline · compare against production ·<br>measure false suppression and missed harm separately ·<br>deploy with a version and a rollback |

Client comments are **operational customer data by default, not an automatic training set**. Only explicit Corrections or approved annotations can become dataset candidates, subject to client permission. Evaluation samples never enter training data.

## 8. Measuring honestly

| Metric                 | Measures                                             |
|------------------------|------------------------------------------------------|
| False Suppression Rate | Legitimate institutional criticism marked for hiding |
| Missed Harm Rate       | Targeted abuse the model cleared                     |
| Suppression Rate       | What actually got hidden                             |
| Disagreement Rate      | How often our own annotators differ                  |

**FSR is the headline** because it is policy-independent. **MHR is always reported alongside it**, so neither can be improved by quietly sacrificing the other — an abstention improves one and worsens the other by exactly as much.

A rate with no denominator says *what is missing*, rather than showing zero. Zero would claim perfection where there is only absence of data, about the one thing this product says it is best at.

---

## 9. What we deliberately cannot do

---

**No pre-publication filtering.** Facebook has no hook for it. The comment is public before we see it.

**No editing another user's comment.** PII cannot be masked in place; the only options are hide or leave.

**No deletion, no banning.** We hide. Hidden comments remain visible to their author and to page admins, so reach drops without speech being erased. Hiding is reversible; deletion is not, and the entire audit model depends on being able to undo.

**No customer self-onboarding onto arbitrary Pages.** The Meta app is in development mode, so the Graph API only works where the same person holds both Page-admin and app-admin roles. Serving arbitrary pages requires App Review, business verification, a hosted privacy policy and a data-deletion endpoint.

**No image understanding.** Toxic content posted as a screenshot is invisible to a text classifier. Khmer OCR is unreliable — stacked consonants, diacritics, no word spacing — so rather than ship a half-working pipeline we measure how often image-based evasion occurs and report the figure.

---

## 10. Roadmap

---

### Next

1. Comment review detail — original post, parent comment, thread. Context is what separates an insult from a quote, and the model cannot be judged fairly without it either.
2. Full moderation history, not only the latest action.
3. Platform administration: workspaces, users, fleet health, audit log.

### Then

4. Replaceable ingestion source interface, then real Facebook ingestion.
5. Telegram bot for alerts — a client should not have to remember to open a dashboard. This is the retention mechanism, and sign-up through it is close to free once it exists.
6. Seed dataset, annotation tooling, baseline Khmer model.

### Later

7. Shadow mode on live client traffic.

8. Controlled automatic hiding, only for repeatedly validated categories, only with explicit customer approval, never for institution-directed criticism.
9. Suggested replies with human approval; Messenger and Instagram.

---

## Appendix A — Technology

---

| Layer    | Choice                                               | Why                                               |
|----------|------------------------------------------------------|---------------------------------------------------|
| Backend  | FastAPI, Python 3.12, uv                             | Classifier and evaluation work is Python-oriented |
| Database | PostgreSQL 16, asyncpg                               | Transactional moderation history                  |
| Contract | OpenAPI 3.1, byte-stable artifact                    | Contract drift fails deterministically            |
| Frontend | React 19, TypeScript strict, Vite                    | Generated client, no hand-maintained types        |
| Testing  | pytest, Vitest, Playwright                           | HTTP as the single backend seam; real Postgres    |
| Hosting  | Vercel (frontend), Render Singapore (API + database) | Closest region to Cambodia                        |

---

## Appendix B — API surface

|        |                                                   |                              |
|--------|---------------------------------------------------|------------------------------|
| GET    | /api/v1/health                                    | database-aware probe         |
| POST   | /api/v1/auth/signin · telegram                    | sessions                     |
| GET    | /api/v1/auth/me · providers                       |                              |
| POST   | /api/v1/auth/signout                              |                              |
| POST   | /api/v1/pilot-requests                            | public pilot request         |
| GET    | /api/v1/admin/pilot-requests                      | platform admin only          |
| POST   | /api/v1/admin/pilot-requests/{id}/decision        |                              |
| GET    | /api/v1/setup-invitations/{token}                 | one-time owner setup         |
| POST   | /api/v1/setup-invitations/{token}                 |                              |
| GET    | /api/v1/comments                                  | paginated work list          |
| GET    | /api/v1/comments/summary                          | workspace-wide counts        |
| POST   | /api/v1/comments/{id}/actions                     | hide · leave · unhide        |
| POST   | /api/v1/comments/{id}/corrections                 | label correction             |
| GET    | /api/v1/facebook/connection                       | current Page connection      |
| DELETE | /api/v1/facebook/connection                       | disconnect Page              |
| POST   | /api/v1/facebook/oauth/start                      | begin Facebook authorization |
| GET    | /api/v1/facebook/oauth/sessions/{state}           | list authorized Pages        |
| POST   | /api/v1/facebook/oauth/sessions/{state}/selection |                              |
| POST   | /api/v1/facebook/connections/manual               | advanced Page-token setup    |
| GET    | /api/v1/team                                      | members and invitations      |
| POST   | /api/v1/team/invitations                          | owner only                   |
| GET    | /api/v1/team/invitations/{token}/preview          | public                       |
| POST   | /api/v1/team/invitations/{token}/accept           |                              |
| DELETE | /api/v1/team/members/{id}                         | owner only                   |
| GET    | /api/v1/settings                                  |                              |
| PATCH  | /api/v1/settings/workspace                        | owner only                   |
| PATCH  | /api/v1/settings/me                               |                              |

## Appendix C — Accepted decisions

| #     | Decision                                                                          |
|-------|-----------------------------------------------------------------------------------|
| D-015 | FastAPI backend, React 19 frontend, PostgreSQL                                    |
| D-016 | Contract-first delivery; simulation only at the network boundary                  |
| D-018 | Open sign-up into a sandbox workspace; capability is gated, not registration      |
| D-019 | Authored Khmer sample comments are accepted in the prototype and labelled as such |
| D-020 | One work list carrying a surfacing reason, rather than three separate lanes       |
| D-021 | Unfamiliar language is a distinct surfacing reason, not an abstention             |
| D-022 | Bearer tokens rather than cookies, because the two halves are cross-site          |
| D-023 | Design, then contract, then whichever half carries the risk                       |

*KCMS is an early prototype. Every capability described as built is deployed and tested; everything else is marked as not built. No accuracy claim is made about Khmer language understanding, because the trained model does not exist yet.*